User Defined Primitives

1 Introduction

User-defined primitives (UDPs) describe custom Verilog logic with a lookup table. A table row maps an input pattern to an output value. UDPs can describe combinational functions with a truth table or sequential functions with a state table. The tables can also use edge and don't-care notation to express input conditions compactly.

A combinational UDP table row has this general form:

$$\texttt{input}_1\ \texttt{input}_2\ \ldots\texttt{input}_N\ :\ \texttt{output};$$

A sequential UDP row additionally gives the present state and next state:

$$\texttt{input}_1\ \texttt{input}_2\ \ldots\texttt{input}_N\ :\ \texttt{present_state}\ :\ \texttt{next_state};$$

The output terminal is always the first terminal in the primitive declaration, and a UDP has exactly one scalar output. Its input terminals are scalar and are declared with `input`. The order of the input values in every table row must match the order of the declared input terminals. A combinational output is declared with `output`; a sequential output is declared with `output reg`.

UDPs model logic functionality, not timing or process technology. A block with multiple outputs cannot be represented by one UDP; it can instead be modeled as a module or as multiple UDPs, one for each output. Simulators typically implement a UDP as a lookup table.

2 Combinational UDPs

A combinational primitive starts with `primitive`, contains a `table` ... `endtable` block, and ends with `endprimitive`. Each row lists its input values followed by the output after a colon.

2.1 Full-adder sum and carry

The sum output is 1 when an odd number of the three inputs are 1. The carry output is 1 when at least two inputs are 1. The first carry table spells out every input combination; the second uses `?` to combine rows with the same result.

```
// Full adder sum generation using UDP
primitive udp_sum (sum, a, b, c);
  input a, b, c;
  output sum;
  table
  // a b c : sum
     0 0 0 : 0;
     0 0 1 : 1;
     0 1 0 : 1;
     0 1 1 : 0;
     1 0 0 : 1;
     1 0 1 : 0;
     1 1 0 : 0;
     1 1 1 : 1;
  endtable
endprimitive
```

Listing 1: Full-adder sum generation

```
// Full adder carry generation using UDP
primitive udp_cy (cout, a, b, c);
  input a, b, c;
  output cout;
  table
  // a b c : cout
     0 0 0 : 0;
     0 0 1 : 0;
     0 1 0 : 0;
     0 1 1 : 1;
     1 0 0 : 0;
     1 0 1 : 1;
     1 1 0 : 1;
     1 1 1 : 1;
  endtable
endprimitive
```

Listing 2: Full-adder carry, with every input combination

```
// Full adder carry generation using don't-care inputs
primitive udp_cy (cout, a, b, c);
  input a, b, c;
  output cout;
  table
  // a b c : cout
     0 0 ? : 0;
     0 ? 0 : 0;
     ? 0 0 : 0;
     1 1 ? : 1;
     1 ? 1 : 1;
     ? 1 1 : 1;
  endtable
endprimitive
```

Listing 3: The same carry function, using don't-care inputs

In these binary examples, `?` is a wildcard in an input column and matches either 0 or 1. For example, `1 1 ? : 1;` covers both rows where `a` and `b` are 1, regardless of `c`.

2.2 Instantiating UDPs

A UDP can be instantiated in a module like another primitive. The connection order follows the terminal order in the UDP declaration.

```
// Instantiating UDPs: a full adder description
module full_adder (sum, cout, a, b, c);
  input a, b, c;
  output sum, cout;

  udp_sum SUM (sum, a, b, c);
  udp_cy  CARRY (cout, a, b, c);
endmodule
```

Listing 4: A full adder built from two UDP instances

2.3 Four-input AND and OR

The `?` entries let each table list the cases that determine its output without repeating every combination. For the AND, any zero input forces the output to zero; for the OR, any one input forces it to one.

```
// A 4-input AND function
primitive udp_and4 (f, a, b, c, d);
  input a, b, c, d;
  output f;
  table
  // a b c d : f
     0 ? ? ? : 0;
     ? 0 ? ? : 0;
     ? ? 0 ? : 0;
     ? ? ? 0 : 0;
     1 1 1 1 : 1;
  endtable
endprimitive
```

Listing 5: Four-input AND

```
// A 4-input OR function
primitive udp_or4 (f, a, b, c, d);
  input a, b, c, d;
  output f;
  table
  // a b c d : f
     1 ? ? ? : 1;
     ? 1 ? ? : 1;
     ? ? 1 ? : 1;
     ? ? ? 1 : 1;
     0 0 0 0 : 0;
  endtable
endprimitive
```

Listing 6: Four-input OR

2.4 Four-to-one multiplexer

This table lists the select inputs first, followed by data inputs `i0` through `i3`. Each pair of select values enables one data input; the other data columns are don't-cares for that row.

```
// A 4-to-1 multiplexer
primitive udp_mux41 (f, s0, s1, i0, i1, i2, i3);
  input s0, s1, i0, i1, i2, i3;
  output f;
  table
  // s0 s1 i0 i1 i2 i3 : f
     0  0  0  ?  ?  ?  : 0;
     0  0  1  ?  ?  ?  : 1;
     1  0  ?  0  ?  ?  : 0;
     1  0  ?  1  ?  ?  : 1;
     0  1  ?  ?  0  ?  : 0;
     0  1  ?  ?  1  ?  : 1;
     1  1  ?  ?  ?  0  : 0;
     1  1  ?  ?  ?  1  : 1;
  endtable
endprimitive
```

Listing 7: A 4-to-1 multiplexer

3 Sequential UDPs

A sequential UDP declares its output as `reg`. Its table rows include the present state and the next state, separated by colons. A sequential UDP can also set its initial state with an `initial` statement.

Table symbols: in these binary examples, `?` acts as a wildcard in input and present-state columns, but not in the output or next-state column. The symbol `-` means no change and is used only in the next-state field. The shortcuts `r` and `f` represent rising `(01)` and falling `(10)` edges, respectively; `*` represents any value change.

3.1 Level-sensitive D latch

This latch is transparent while `clk` is high and `clr` is low: the output follows `d`. When `clk` is low, the `-` entry holds the previous state. A high `clr` clears the output, and the optional initial statement starts it at zero.

```
// A level-sensitive D type latch
primitive Dlatch (q, d, clk, clr);
  input d, clk, clr;
  output reg q;

  initial
    q = 0;  // This is optional

  table
  // d clk clr : q : q_new
     ?  ?   1  : ? : 0;  // latch is cleared
     0  1   0  : ? : 0;  // latch is reset
     1  1   0  : ? : 1;  // latch is set
     ?  0   0  : ? : -;  // retains previous state
  endtable
endprimitive
```

Listing 8: A level-sensitive D-type latch

3.2 T flip-flop and ripple counter

The T flip-flop toggles on the falling edge of `clk` when not cleared. Its state table gives separate rows for each current state, while the final row ignores a rising clock edge. The source image labels the counter wrapper as a primitive, but it instantiates UDPs and has a six-bit output, so it is written here as a module. The counter connects six flip-flops in a ripple arrangement: each stage after the first uses the previous stage's output as its clock.

```
// A T flip-flop
primitive TFF (q, clk, clr);
  input clk, clr;
  output reg q;

  table
  // clk  clr : q : q_new
     ?     1  : ? : 0;  // FF is cleared
     ?    (10): ? : -;  // ignore -ve edge of "clr"
    (10)   0  : 1 : 0;  // FF toggles at -ve edge of "clk"
    (10)   0  : 0 : 1;  // FF toggles at -ve edge of "clk"
    (0?)   0  : ? : -;  // ignore +ve edge of "clk"
  endtable
endprimitive
```

Listing 9: A T flip-flop

```
// Constructing a 6-bit ripple counter using T flip-flops
module ripple_counter (count, clk, clr);
  input clk, clr;
  output [5:0] count;

  TFF F0 (count[0], clk, clr);
  TFF F1 (count[1], count[0], clr);
  TFF F2 (count[2], count[1], clr);
  TFF F3 (count[3], count[2], clr);
  TFF F4 (count[4], count[3], clr);
  TFF F5 (count[5], count[4], clr);
endmodule
```

Listing 10: A six-bit ripple counter using T flip-flops

3.3 Negative-edge JK flip-flop

The JK table responds to the falling edge `(10)` of the clock when clear is inactive. The four J/K combinations hold, reset, set, or toggle the state; the two toggle rows make the next value depend on the present state.

```
// A negative edge sensitive JK flip-flop
primitive JKFF (q, j, k, clk, clr);
  input j, k, clk, clr;
  output reg q;

  table
  // j k clk  clr : q : q_new
     ? ?  ?    1  : ? : 0;  // clear
     ? ?  ?   (10): ? : -;  // ignore ... no change
     0 0 (10)  0  : ? : -;  // no change
     0 1 (10)  0  : ? : 0;  // reset condition
     1 0 (10)  0  : ? : 1;  // set condition
     1 1 (10)  0  : 0 : 1;  // toggle condition
     1 1 (10)  0  : 1 : 0;  // toggle condition
     ? ? (01)  0  : ? : -;  // no change
  endtable
endprimitive
```

Listing 11: A negative-edge-sensitive JK flip-flop

3.4 Positive-edge SR flip-flop

The SR example responds to the rising edge `(01)` of the clock. With clear inactive, `s=0,r=0` holds, `s=0,r=1` resets, and `s=1,r=0` sets. The `s=1,r=1` row marks the combination as invalid with an unknown next state, `x`.

```
// A positive edge sensitive SR flip-flop
primitive SRFF (q, s, r, clk, clr);
  input s, r, clk, clr;
  output reg q;

  table
  // s r clk  clr : q : q_new
     ? ?  ?    1  : ? : 0;  // clear
     ? ?  ?   (10): ? : -;  // ignore ... no change
     0 0 (01)  0  : ? : -;  // no change
     0 1 (01)  0  : ? : 0;  // reset condition
     1 0 (01)  0  : ? : 1;  // set condition
     1 1 (01)  0  : ? : x;  // invalid condition
     ? ? (10)  0  : ? : -;  // ignore ... no change
  endtable
endprimitive
```

Listing 12: A positive-edge-sensitive SR flip-flop